

Atividade 02 – Criando um Jogo

Andrei Formiga

May 16, 2026

1. Introdução

Nesta atividade vamos criar um jogo simples, primeiramente com uma versão mais básica e depois adicionando novas funcionalidades, tudo feito com o uso de algum agente de codificação. O objetivo é analisar a experiência de desenvolvimento e os resultados obtidos.

2. O jogo

O jogo a ser feito deve ser simples, mas que permita a adição de várias funcionalidades em cima da base do jogo.

Minha sugestão é o jogo Tetris, que praticamente todo mundo deve conhecer. Os grupos podem escolher criar algum outro jogo, mas de preferência devem entrar em contato para combinar as mudanças na atividade.

A plataforma sugerida para o jogo é HTML/JavaScript (ou TypeScript), para funcionar no browser. Isso facilita o desenvolvimento e testes, sem precisar instalar nada extra, mas também deve ser mais fácil para os modelos, que foram treinados com muito código JavaScript (incluindo código de jogos). Os grupos que quiserem podem tentar desenvolver usando algum *engine* específico para jogos como Unity ou Godot, mas minha expectativa é que os modelos consigam ajudar menos nesse caso.

3. Primeira Etapa: a base do jogo

Inicialmente o objetivo é criar uma versão básica do jogo, com seus elementos principais: o campo de jogo, todos os (sete) tipos de tetraminós, controle das peças (movimento direita/esquerda, rotação, aceleração da queda), peças caindo com o tempo, detecção de colisão, a mecânica de limpar linhas completas e a condição de fim de jogo (quando não é possível mais adicionar uma peça no meio da tela pois alguma das posições já está preenchida). O jogo também deve mostrar a próxima peça que vai aparecer.

O grupo pode começar com um prompt muito amplo, como “crie um jogo de Tetris em JavaScript e HTML”, ou pode ser mais detalhado. Um prompt mais detalhado vai dar ao grupo mais controle de como as coisas são feitas e talvez evite retrabalho. Com um prompt mais vago, é difícil prever o quê o modelo vai fazer, talvez inclua coisas que não são previstas para esta primeira etapa, e talvez falte algumas características necessárias.

É importante guardar os prompts que foram utilizados na criação do projeto para o relatório, como será detalhado mais a frente.

4. Segunda Etapa: níveis e pontuação

Com o jogo base funcionando, nesta etapa vamos adicionar níveis ao jogo, da forma como geralmente é feita em jogos de Tetris: após um certo número de linhas limpas, o jogo sobe de nível e as peças passam a cair de maneira mais rápida. Nesta segunda etapa também deve ser adicionada a pontuação do jogador. O grupo pode decidir sobre as regras de pontuação que vai usar, e/ou pode consultar as regras de várias versões já existentes. A pontuação deve ser mostrada na tela durante o jogo, assim como o nível atual e a próxima peça.

Opcional: adicionar a funcionalidade de pontuações mais altas (*high scores*), mostrando na tela do jogo a pontuação mais alta obtida até o momento.

5. Terceira Etapa: tela de abertura

Para finalizar, nesta etapa o jogo deve começar com uma tela de abertura e esperar o usuário interagir de alguma forma (com botão do mouse, tecla do teclado, etc) para começar o jogo. Ao fim de uma jogada (*game over*), deve ser mostrado “GAME OVER” na tela e o jogo deve retornar para a tela de abertura.

Opcional: se o grupo tiver implementado *high scores* na segunda etapa, pode criar alguma forma de ver as pontuações, seja depois de um tempo esperando na tela de abertura ou com alguma opção antes de começar o jogo.

Opcional: alguns jogos, ao passarem muito tempo na tela de abertura, mudam para o chamado *attract mode*, mostrando uma demonstração do jogo. O *attract mode* deve terminar se o usuário interagir de alguma forma. Isso também pode ser combinado com a tela de *high scores* no loop tradicional dos jogos de *arcade*: tela de abertura, depois de algum tempo mostrar o *attract mode*, depois de mais algum tempo mostra a tela de *high scores*, e depois volta para a tela de abertura e continua o loop.

6. Quarta Etapa: outras adições

Neste ponto o grupo pode adicionar mais algumas funcionalidades extras ao jogo, tornando-o mais diferenciado. Seguem algumas sugestões.

6.1. Persistência das pontuações

Se o jogo tem suporte a *high scores*, pode ser uma boa ideia gravar as pontuações para que elas sejam recuperadas quando o usuário abrir novamente o jogo.

No browser, uma possibilidade simples é usar a capacidade de armazenamento local (*local storage*), que é facilmente acessível usando JavaScript/TypeScript. Em outras plataformas as pontuações podem ser gravadas e lidas de arquivos locais.

6.2. Suporte a estilos visuais

Um jogo em HTML e JavaScript (ou TypeScript) provavelmente vai usar o elemento Canvas para mostrar seus gráficos. Em um canvas é fácil desenhar formas como quadrados e retângulos, e isso é suficiente para desenhar as peças (tetraminós) e o campo do jogo. Mas o jogo fica visualmente mais interessante com peças que tenham mais detalhes.

O programa do jogo pode ter suporte ao uso de imagens para desenhar as peças, o campo de jogo e quaisquer outros elementos que precisam ser mostrados (incluindo as fontes). Seria até possível ter diferentes temas visuais e permitir ao usuário trocar o tema durante o jogo.

Deve ser fácil achar gráficos para jogos de Tetris na internet, ou gerar com algum modelo de geração de imagens.

6.3. Multiplayer

Várias versões de Tetris têm a funcionalidade multi-jogador, permitindo pelo menos dois jogadores ao mesmo tempo. Com dois jogadores o jogo pode ser apenas uma competição de pontuação, mas também pode ter outras formas de competição. Por exemplo, quando um jogador limpa mais de uma linha ao mesmo tempo, isso cria um número de linhas de “lixo” no campo do outro jogador, tornando seu jogo mais difícil.

O suporte a multi-jogador deve ser local. Adicionar a capacidade de multi-jogador por rede é bem mais complicado de fazer corretamente.

Também podem ser criadas formas de multiplayer assíncrono, com um jogador mandando um desafio para outro. O desafio pode ser alguma pontuação que deve ser alcançada, ou número de linhas limpas, ou número de tetris (quando o jogador limpa 4 linhas ao mesmo tempo).

6.4. Efeitos sonoros e música

Adicionar efeitos e sonoros e música ao jogo ajuda muito a criar uma atmosfera, e algumas músicas de versões de Tetris (como a do jogo original de Game Boy[®]) ficaram famosas.

O grupo pode adicionar apenas efeitos sonoros ou apenas música, ou os dois.

6.5. Efeitos visuais

Quando o jogador remove uma linha do campo, o mais fácil é simplesmente fazer a linha sumir, mas outra possibilidade seria criar alguma animação ou efeito visual que mostre a(s) linha(s) sendo eliminada(s) de forma mais visível.

6.6. Outras adições

Qualquer outra adição ou funcionalidade que o grupo quiser implementar no jogo, usando agentes.

7. Relatório

O grupo deve entregar o código do projeto gerado, mas o mais importante é o relatório produzido pelo grupo. O relatório deve ter pelo menos três partes: uma sobre experiência prévia do grupo, o relato do desenvolvimento do projeto, e as conclusões e comentários.

7.1. Experiência prévia

Inicialmente é importante estabelecer quanta experiência o grupo já tinha com projetos similares.

Qual a experiência prévia do grupo com a linguagem e plataforma do jogo (no caso sugerido, HTML/JavaScript/TypeScript)? Qual a experiência prévia do grupo com o domínio do projeto (desenvolvimento de jogos)?

7.2. Relato do desenvolvimento

O grupo deve relatar como ocorreu o desenvolvimento do projeto, separado nas etapas que foram sugeridas neste guia. É importante especificar quais foram as ferramentas de agentes utilizadas, incluindo qual a estrutura de agentes e qual o modelo (ou modelos, em caso de mais de um) utilizados.

Não é necessário incluir todos os prompts utilizados mas é bom incluir pelo menos os principais. Caso alguma etapa tenha precisado de retrabalho e vários prompts auxiliares para correções, é importante mencionar que isso aconteceu, mas não precisa listar cada prompt individualmente.

É interessante analisar partes do código gerado pelo agente de tempos em tempos (por exemplo, ao final de cada etapa), dentro das possibilidades do grupo (que talvez não tenha conhecimento da linguagem).

7.3. Conclusões e comentários

Aqui o grupo deve comentar sobre sua experiência desenvolvendo o projeto com agentes de codificação, e tecer comentários sobre o uso dessas ferramentas e seu próprio aprendizado.

Algumas perguntas que podem ser respondidas:

- Para quem não tinha experiência prévia com as tecnologias usadas e/ou o desenvolvimento de jogos, vocês acham que essa falta de experiência teve impacto no desenvolvimento do projeto? Caso fossem continuar esse projeto, pensam que seria necessário estudar mais sobre as tecnologias e/ou sobre desenvolvimento de jogos em geral?
- O que aprenderam durante o desenvolvimento do projeto? Conseguiriam fazer um projeto similar sem o uso de agentes de codificação?
- Olhando o código da versão final, acham que o resultado é legível? Seria fácil de manter no futuro? Acham que seria necessário refatorar o código?
- Já comentamos sobre o conceito de *déficit de compreensão* (*comprehension debt*). Sentem que entendem o projeto que foi gerado? Se fosse necessário fazer mais modificações, conseguiriam fazer alterações sem usar agentes? Como conseguem julgar se uma modificação pedida ao agente foi bem-sucedida?

Além de responder essas perguntas, os grupos podem fazer quaisquer outros comentários que acharem interessantes sobre o processo de desenvolvimento deste projeto.

8. Apresentação

O grupo deve mostrar os resultados do projeto ao final de cada etapa, fazendo os comentários mais importantes sobre o que aconteceu durante o processo de desenvolvimento. No final, comentar sobre a experiência do grupo no projeto e quaisquer outros comentários que julgarem importantes.